

Lunyad

Backyard-Ultra Live-Dashboard

FSST-Projekt – Projektdokumentation

1. Projektidee & -konzept

Lunyad ist ein Live-Dashboard für ein Backyard-Ultra-Rennen.

Bei einem Backyard Ultra startet jede volle Stunde eine neue Runde. Wer die feste Strecke nicht rechtzeitig schafft, scheidet aus – der Letzte gewinnt.

Strava Beacon zeigt zwar Live-Position und Distanz jedes Läufers, rechnet aber nicht aus, wer seine Runde geschafft hat. Genau das macht unsere App:

- Sie zeigt mehrere Strava-Beacon-Livekarten gleichzeitig (z. B. am Beamer).
- Sie liest automatisch Name, Distanz und Zeit aus.
- Sie berechnet Runden, Pace und Ausscheidungen selbst.
- Unten läuft ein Info-Streifen für alle Zuschauer (Countdown, Status, Uhr).

2. Projektplanung (Feature-Liste)

Priorisierung: Muss (wichtig), Soll (geplant), Kann (Extra).

Feature	Priorität
Mehrere Livekarten anzeigen (1–8 Felder)	Muss
Backyard-Konfiguration (Startzeit, Rundenzeit, Strecke)	Muss
Runden über Timer steuern + Countdown	Muss
Ausgeschiedene Läufer erkennen und anzeigen	Muss
Daten automatisch aus Strava auslesen	Muss
Info-Streifen (Status, Fortschrittsbalken, Ortszeit)	Soll
Pace und Rundenzeit pro Person	Soll
Rundenzeit-Diagramm pro Läufer	Soll
Hell-/Dunkelmodus	Kann
Live-Vollbildmodus	Kann
Eigenes Icon / Start per Verknüpfung	Kann

3. Technologie Stack

- JavaScript – die gesamte Logik
- HTML & CSS – Oberfläche und Design
- Electron – macht aus Web-Technik eine Desktop-App
- Node.js – Laufzeitumgebung
- Strava Beacon – Datenquelle (Live-Standort)

- Tools: Visual Studio Code, Git, Claude, Cursor

4. Teamauflistung

- Fenkart, Benedikt
- Sukhrukov, Maksym

5. KI-Einsatz & Vorgehen

Wir haben das Projekt bewusst mit Unterstützung von Künstlicher Intelligenz umgesetzt, dabei aber großen Wert darauf gelegt, den Code selbst zu verstehen. Unser Vorgehen in Stichpunkten:

- Modell: Claude Opus 4.8 als Programmier-Assistent.
- Iterativ gearbeitet: die App Schritt für Schritt aufgebaut, Feature für Feature – nicht alles auf einmal.
- Klare, präzise Prompts: gewünschte Funktion und erwartetes Verhalten genau beschrieben (z. B. „die Rundenzeit läuft als echte Sekundenuhr ab dem Rundenstart und friert beim Streckenziel ein“).
- Geprüft & getestet: nach jedem Schritt den Code gelesen, die App gestartet und ausprobiert.
- Nachgeschärft: bei falschen Ergebnissen den Prompt verbessert oder gezielt Korrekturen verlangt.
- Reihenfolge: vom Grundgerüst (mehrere Karten anzeigen) über die Strava-Auswertung bis zur kompletten Backyard-Logik.
- Auch für Nebenaufgaben: App-Icon, Dokumentation und Präsentation.
- Verantwortung bei uns: Die KI liefert nur Vorschläge – Entscheidungen und Verständnis bleiben bei uns, damit wir den Code selbst erklären können.